

BEGINNER LEARNING GUIDE

Debugging Claude-Based AI Agents

A practical guide for AI Customer Success

The core mindset Debug the whole system, not just Claude. Your job is to find where the first mistake happened, collect clear evidence, and help the right owner act.

This guide focuses on what you are most likely to own: understanding the report, locating the conversation and trace, reproducing the problem, identifying the likely failure point, checking customer impact, validating fixes, and communicating a crisp handoff.

You do not need to become an AI engineer. You do need to become comfortable explaining: “Claude received this information, made this decision, called this tool, and the problem started here.”

Quick orientation

Question	Short answer
What is an agent?	Claude plus instructions, data, tools, and rules that let it answer questions or take actions.
What is debugging?	Finding the first point where actual behavior stopped matching expected behavior.
What is a trace?	A step-by-step record of one request: inputs, model calls, tool calls, outputs, errors, timing, and usage.
What is your main value?	Turning a vague customer complaint into reproducible evidence, a likely cause, impact, and a verified outcome.
When do you hand off?	When the issue requires code, infrastructure, access-policy, security, or product-priority changes.

1. What agent debugging means

Agent debugging means figuring out why the agent did not behave as intended. Think of the agent as a new employee: it receives a request, follows instructions, looks up information, uses tools, and explains a result. A mistake can happen at any step.

Your goal Compare what should have happened with what actually happened, then locate the earliest meaningful difference.

A simple example

A customer asks, “Has my order shipped?” The order system returns “label created.” Claude answers, “Yes, it shipped.”

Trace step	What happened	Debug question
Customer request	Asked whether the order shipped.	Was the request understood correctly?
Tool call	Claude looked up the right order.	Were the tool and order ID correct?
Tool result	Status was “label created.”	Was the source data correct and current?
Interpretation	Claude treated “label created” as “shipped.”	Were the instructions or status definitions unclear?
Final answer	Customer received incorrect confirmation.	Was there a safe fallback or escalation rule?

Likely fix: clarify that only “shipped” or “in transit” confirms carrier possession. Then test several wording variations and nearby statuses before calling the fix successful.

2. Common failure sources

Failure source	What it looks like	What AI Customer Success checks
User request	Ambiguous wording, missing identifier, unusual phrasing.	Capture the exact wording and needed account context.
Instructions / prompt	Policy is missing, vague, conflicting, or outdated.	Compare behavior with the written rule; flag the precise gap.
Conversation context	Earlier turns are missing, too long, or misunderstood.	Read the full thread; note the first turn where context drifted.
Knowledge / retrieval	Wrong, stale, irrelevant, or missing documents.	Identify the source used, its date, and what should have been retrieved.
Tool choice	Claude uses the wrong tool or skips a needed tool.	Inspect the available tool names/descriptions and the selected tool.
Tool inputs	Wrong customer ID, date, amount, or required field.	Compare sent arguments with the customer/account facts.
Tool result	API error, timeout, empty result, or bad upstream data.	Record the raw result and separate tool failure from model interpretation.
Permissions	Access denied—or an action was allowed when it should not be.	Confirm user, role, scope, approval, and intended policy.
Model response	Correct facts but unclear, overconfident, or policy-breaking answer.	Compare final wording with expected tone, certainty, and policy.
Workflow logic	Loops, repeated calls, no stop condition, bad handoff.	Count retries and confirm escalation/stop rules fired.
Operations	Slow response, high cost, intermittent availability.	Record timing, token/tool usage, frequency, and customer impact.

3. Tracing tools in a Claude setup

The tracing tool usually depends more on how the agent was built than on the fact that it uses Claude. Ask your company: “Where do I find the production trace for a conversation ID?”

Tool	Best beginner mental model	What you are likely to use it for
LangSmith	Agent-focused trace viewer and evaluation workspace.	Open a conversation or trace; inspect runs, inputs, outputs, tool calls, errors, timing, tokens, metadata; add examples to datasets.
Langfuse	Open-source LLM observability, prompt, and evaluation platform.	Inspect trace trees and sessions; compare prompt versions; view token, cost, latency, and evaluation data.
Datadog Agent Observability	AI traces inside a broader production-monitoring platform.	Connect a bad AI response with errors, latency, infrastructure, usage, safety, and operational trends.
Anthropic Console	Prompt playground and evaluation workspace for Claude.	Try prompts, create/import test cases, compare prompt versions, grade outputs, and rerun a suite.
Company dashboard / ticketing	The team's operational source of truth.	Find conversation IDs, account context, incidents, owners, trace links, and fix status.

Important distinction A trace shows the request's journey. A log often shows technical events and errors. A metric shows a trend, such as error rate, latency, or cost over time.

4. Step-by-step debugging workflow

1. **Clarify the expected behavior.** Write one plain sentence: "Given X, the agent should do Y." Confirm the policy or source of truth.
2. **Capture the actual behavior.** Save the exact customer request, response, timestamps, conversation ID, account conditions, and screenshots or links.
3. **Assess urgency and safety.** Escalate immediately for privacy exposure, unauthorized actions, financial harm, security risk, or broad customer impact.
4. **Find the trace.** Search by conversation/request ID, customer/account identifier, time window, environment, or trace ID.
5. **Read from left to right.** Start at the user input. Follow each model, retrieval, and tool step. Stop at the first step that differs from expectation.
6. **Classify the likely cause.** Instruction, context, retrieval, tool selection, tool input, tool result, permissions, model wording, workflow, or operations.
7. **Reproduce safely.** Use a test account or approved environment. Start with the exact request, then vary only one condition at a time.
8. **Collect comparison evidence.** Include one failing run and, when possible, one successful run that should behave similarly.
9. **Propose or route the smallest fix.** Customer Success may improve examples or test cases; engineering owns code; security owns access policy; PM owns product tradeoffs.

10. **Run evals and regression checks.** Test the original case, paraphrases, edge cases, neighboring intents, and actions that must remain blocked.
11. **Close the loop.** Record what changed, evidence that it worked, remaining risk, monitoring plan, and customer-facing update.

5. How to read a trace

Trace-reading rule Do not jump straight to the final answer. Find the first incorrect input, decision, or result; later errors may only be consequences.

Inspect	Questions to ask	Evidence to capture
Root input	What exactly did the user say? Which IDs, dates, or account facts were available?	Exact text; environment; timestamp; user/account context.
System/developer instructions	What rules did Claude receive? Were they current and consistent?	Relevant instruction excerpt and version—not hidden reasoning.
Retrieved context	Which documents or records were returned? Were they relevant and fresh?	Source names, timestamps, snippets, missing expected source.
Tool decision	Did Claude call the right tool at the right moment?	Available tools; chosen tool; skipped tool; ordering.
Tool arguments	Were the structured inputs correct and complete?	Tool name and argument values, with sensitive data minimized.
Tool result	Did the tool succeed? What raw data or error came back?	Status/error code, result fields, timing, retry count.
Next model step	Did Claude interpret the result correctly?	Result-to-response mismatch or unsupported claim.
Final response/action	Was it correct, clear, safe, and within permission?	Actual output/action versus expected behavior.
Performance	Where was time or cost spent? Was work repeated?	Total and per-step latency, tool calls, tokens, errors.

Useful comparison: open one “good” trace next to the failing trace. Look for the first structural difference—different prompt version, missing document, different tool, changed arguments, error, retry, or permission result.

6. Reproduction and evals

Reproduce before changing

- Use the exact customer wording first.
- Match the relevant conditions: account type, permissions, data state, locale, channel, and agent version.

- Use safe test data. Never repeat a risky action on a real customer account just to prove it fails.
- Change one variable at a time so you know what affected the result.
- Record whether the failure is consistent, intermittent, or no longer reproducible.

Turn the bug into a small eval set

Test type	Example	Why it matters
Original failure	“Has my order shipped?” with status “label created.”	Confirms the reported bug is fixed.
Paraphrases	“Is it on the way?” / “Does the carrier have it?”	Checks that the fix is not tied to one wording.
Positive case	Status is “in transit.”	Ensures valid shipping confirmation still works.
Neighboring case	Status is “delivered” or “delayed.”	Checks nearby statuses and wording.
Missing data	Order ID absent or tool returns empty.	Checks clarification and fallback behavior.
Permission case	User asks to cancel another person’s order.	Checks that unsafe actions stay blocked.
Failure case	Order tool times out.	Checks retries, honest uncertainty, and escalation.

A fix is not one successful test AI output can vary. Re-run multiple cases and compare the new version with the prior baseline. Confirm both improvement and no important regressions.

7. Tool calls and tool descriptions

Claude does not directly execute most company tools. It produces a structured request containing the tool name and arguments. The application runs the tool and returns a structured result. Claude then uses that result to continue.

Part	Simple meaning	What to inspect
Tool name	The action Claude can choose.	Is it clear and distinct from similar tools?
Description	Instructions that help Claude decide when and when not to use it.	Purpose, boundaries, prerequisites, forbidden uses, return meaning.
Input schema	Required fields and allowed shapes.	Required IDs, types, enums, formats, validation.
Tool call	Claude’s structured request.	Correct tool, arguments, timing, duplicates.

Part	Simple meaning	What to inspect
Tool result	The application's response to Claude.	Success/error, data freshness, empty fields, status semantics.
Final interpretation	Claude's explanation or next action.	Does it follow from the tool result without inventing facts?

Good descriptions reduce ambiguity

Vague	Clearer
check_order	Retrieve the latest order and shipping status for a verified order ID. Use for status questions only. Do not cancel or modify an order.
refund	Create a refund only after eligibility and user authorization have been confirmed. Requires order ID, amount, reason, and approval token.
search_customer	Find a customer record using verified email or account ID. Return possible matches; do not expose records without authorization.

Customer Success contribution: report confusing tool behavior with examples. Engineering contribution: update the tool schema, description, execution code, validation, and error handling.

8. Safety, speed, cost, and reliability

Area	Healthy behavior	Customer Success checks / escalation trigger
Permissions	Least access needed; approval for sensitive actions; correct customer scope.	Who requested it? What role? What data/action? Escalate unauthorized access or over-permission immediately.
Escalation	Agent admits uncertainty, stops after defined failures, and transfers with context.	Did it stop at the right time? Did the human receive the useful context?
Speed	Reasonable total response time; no single unexplained bottleneck.	Record total and per-step latency. Compare with a normal run.
Cost	Necessary model/tool calls; controlled context and token use.	Look for repeated calls, oversized inputs, needless retries, and expensive paths.
Reliability	Consistent results, graceful timeouts, bounded retries, clear fallback.	Estimate frequency; identify affected segment/version; test tool failure and empty-result cases.
Privacy	Sensitive data is minimized, properly scoped, and redacted in tools/logs.	Do not paste secrets or excess customer data into tickets; follow incident policy.

Stop and escalate now Potential data exposure, unauthorized account changes, payment/refund harm, security abuse, repeated unsafe actions, or a high-volume production incident.

9. MCP, in simple terms

MCP (Model Context Protocol) is a standard way for an AI application to connect to outside tools and information. A useful analogy is a standard plug: the AI application and many different systems can communicate through one agreed format instead of inventing a new connection pattern every time.

Piece	Job	Example
MCP host	The AI application where the user works.	A Claude-based application.
MCP client	Maintains the host's connection to one MCP server.	The connection component inside the AI app.
MCP server	Offers tools, resources, or prompts from another system.	A server exposing “find customer” or “read account.”
Connected system	The real source or action system behind the server.	CRM, billing system, database, document store.

What MCP is not MCP is not Claude, and it is not the CRM or database. It is the communication standard connecting the AI application with the server that exposes those capabilities.

How MCP is set up

Setup part	What happens	Typical owner
Choose or build a server	Use an existing server or create one for a private system.	Engineering / platform
Define capabilities	Specify tool names, descriptions, inputs, outputs, and errors.	Engineering, informed by PM and Customer Success examples
Connect the host	Add server address/launch config and transport settings.	Engineering / administrator
Authenticate and authorize	Configure identity, credentials, scopes, approvals, and audit rules.	Security + engineering
Test the connection	Check success, errors, timeouts, data mapping, and safe actions.	Engineering + QA; Customer Success supplies real scenarios

Setup part	What happens	Typical owner
Release and monitor	Roll out, watch failures/latency/usage, and maintain versions.	Operations / engineering
Support in production	Reproduce customer issues, inspect traces, identify the failing boundary, and document impact.	AI Customer Success

Is it automatic? Usually not end to end. A prebuilt remote server may be quick for an administrator to connect and authorize. A private database or company product usually needs engineering work, security review, testing, and ongoing maintenance.

Debugging an MCP-related issue

- Did Claude choose the correct MCP tool?
- Did it send the correct arguments and customer identifier?
- Was the MCP connection available and authenticated?
- Did the MCP server return the correct result—or an error, timeout, or empty response?
- Did the connected system contain correct, current data?
- Did Claude interpret the returned data correctly?
- Should the user have been allowed to access or change that information?

10. Role boundaries and handoffs

Role	Detailed ownership / brief context
AI Customer Success	Owns intake, customer/account context, expected behavior, trace review at an accessible level, safe reproduction, impact and severity, examples/eval cases, clear issue reports, customer communication, fix validation, and follow-through.
AI Product Manager	Sets intended product behavior, priorities, acceptance criteria, tradeoffs, and rollout decisions.
AI / application engineer	Builds prompts/workflows/tools/MCP integrations, changes code and schemas, instruments traces, and fixes technical defects.
Security / privacy	Defines access policy, scopes, approvals, data handling, audit requirements, and incident response.
Operations / SRE	Owns production availability, monitoring, alerting, latency, capacity, incidents, and rollback procedures.
Data / knowledge owner	Maintains source accuracy, freshness, taxonomy, and permissions for business data and documents.

A useful boundary test

You can usually do	Coordinate before doing
Inspect approved conversation and trace data; reproduce in approved test space; write cases and expected outcomes; compare before/after; document findings.	Change production prompts, code, tools, MCP servers, permissions, customer records, refund/payment logic, or monitoring policy without the owner's process.

11. Practical issue-report template

One-line summary [Agent / workflow] did [actual behavior] instead of [expected behavior] for [customer segment / condition].

Customer / account impact: *Who is affected? What did they experience? Include scale if known.*

Severity and urgency: *Low / Medium / High / Critical, with the business or safety reason.*

Environment and version: *Production/test, channel, agent/workflow version, prompt version if visible.*

Date and time: *Include timezone and approximate window.*

Conversation / request / trace IDs: *Add direct links where allowed.*

Exact input: *Customer wording, plus only the account facts needed to reproduce.*

Expected behavior: *One clear sentence and source-of-truth policy/link.*

Actual behavior: *Exact response or action; attach a screenshot if useful.*

Steps to reproduce: *Numbered, minimal, safe steps; note frequency.*

First suspected failure point: *Instruction / context / retrieval / tool / permissions / model response / workflow / operations.*

Trace evidence: *Relevant tool name, arguments, result/error, timing, and the first divergence.*

Comparison evidence: *Link one good trace or expected example if available.*

Risk / workaround: *Privacy, money, safety, scale; safe temporary workaround.*

Suggested next owner: *PM / engineering / security / operations / data owner—and why.*

Acceptance checks: *Original case, paraphrases, positive/negative cases, and regression checks.*

Customer update: *What has been communicated; next update time and owner.*

12. Beginner learning checklist

- ☐ Explain an AI agent as Claude + instructions + context + tools + workflow rules.
- ☐ Explain the difference between a conversation, trace, span/run, log, metric, and eval.
- ☐ Find a production trace from a conversation or request ID.
- ☐ Read a trace from the customer input through the final response.
- ☐ Identify the first divergence from expected behavior.
- ☐ Inspect a Claude tool call: name, description, schema, arguments, result, and interpretation.
- ☐ Separate a model mistake from an upstream data or tool failure.
- ☐ Reproduce an issue safely and change one variable at a time.
- ☐ Create a small eval set with original, paraphrase, positive, negative, edge, permission, and failure cases.
- ☐ Recognize permission, privacy, financial, or security issues that require immediate escalation.
- ☐ Check latency, retries, tokens/cost, error rate, and reliability at a basic level.
- ☐ Explain MCP and identify host, client, server, and connected system.
- ☐ Name the correct owner for prompt/product, code/tool, security, operations, and data-quality changes.
- ☐ Write an issue report another team can act on without re-interviewing you.
- ☐ Validate a fix across multiple tests and communicate the outcome to the customer.

First 30 days: a simple learning path

Week	Practice
1	Learn the company's agent, policies, support workflow, IDs, environments, and escalation rules.
2	Shadow trace reviews. Practice narrating what Claude saw, chose, called, received, and answered.
3	Reproduce past issues in a safe space and turn them into small eval sets.
4	Own a low-risk issue from intake through handoff, validation, documentation, and customer closeout.

Final cheat sheet

When a report arrives Expected → Actual → Impact → Trace → First divergence → Reproduce → Classify → Route → Evals → Close the loop.

If you see...	First place to look
Wrong fact	Retrieved data, tool result, and source freshness.
Wrong action	Tool choice, arguments, permission, and approval rules.
No action	Tool availability/description, missing required input, permission denial, or error.
Confident guess	Missing-data handling, uncertainty instructions, and escalation rule.
Slow response	Per-step timing, repeated calls, retries, retrieval volume, and model latency.
High cost	Token counts, prompt/context size, repeated steps, tool definitions, and model choice.
Works sometimes	Version/environment differences, data conditions, nondeterminism, timeout, or race condition.
Keeps looping	Stop condition, retry count, repeated tool result, and fallback/escalation path.

Sources and further reading

- **Anthropic: How tool use works** — <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- **Anthropic: Console Evaluation Tool** — <https://platform.claude.com/docs/en/test-and-evaluate/eval-tool>
- **Model Context Protocol: Architecture overview** — <https://modelcontextprotocol.io/docs/learn/architecture>
- **LangSmith: Observability concepts** — <https://docs.langchain.com/langsmith/observability-concepts>
- **LangSmith: View traces** — <https://docs.langchain.com/langsmith/view-traces>
- **Langfuse: Observability overview** — <https://langfuse.com/docs/observability/overview>
- **Datadog: Agent Observability** — https://docs.datadoghq.com/llm_observability/

Prepared as a beginner learning reference for AI Customer Success. Product names and interfaces change; follow your company's current documentation, permissions, and incident process.